

Introducción

Docker es una plataforma que permite empaquetar una aplicación junto con sus dependencias en unidades llamadas contenedores. Un contenedor permite que el programa se ejecute de forma más consistente entre equipos diferentes, evitando el problema común de “en mi computador sí funciona”.

En esta guía se explica desde cero qué es Docker, para qué se usa, cómo instalar Docker Desktop en Windows y cómo verificar su funcionamiento. El ejemplo práctico es Dino Run, una aplicación web creada con Python, Flask y SQLite que permite registrar usuarios, iniciar sesión, jugar y conservar puntuaciones.

La guía está diseñada para que el aprendiz comprenda qué ocurre en cada paso: descarga de Docker Desktop, activación de WSL 2, creación del proyecto, construcción de una imagen, ejecución de un contenedor, publicación de puertos, uso de variables de entorno, consulta de logs y ejecución con Docker Compose.

Propósito de la guía: fortalecer la capacidad del aprendiz para instalar Docker, comprender sus conceptos esenciales y ejecutar una aplicación real dentro de contenedores, entregando evidencias verificables del proceso.

Elemento	Descripción
Programa de formación	Desarrollo de software / Programación web / Fundamentos DevOps
Tema	Docker desde cero: instalación, contenedores, imágenes, Dockerfile, puertos, variables de entorno y Compose.
Resultado esperado	El aprendiz instala Docker Desktop, crea una imagen, ejecuta una aplicación en contenedor y documenta evidencias.
Nivel	Básico a intermedio.
Duración estimada	12 a 14 horas.

Objetivos

Objetivo general:

El aprendiz instala y utiliza Docker desde cero para empaquetar, configurar y ejecutar una aplicación web sencilla dentro de un contenedor.

Objetivos específicos:

- Reconocer qué es Docker, qué problema soluciona y para qué se usa en desarrollo de software.
- Instalar Docker Desktop en Windows y validar su funcionamiento con comandos básicos.
- Diferenciar imagen, contenedor, Dockerfile, puerto, volumen y Docker Compose.
- Construir una imagen Docker a partir de una aplicación Flask sencilla.
- Ejecutar y comprobar la aplicación mediante Docker Compose, comprendiendo la publicación de puertos, los montajes y las variables de entorno.
- Usar Docker Compose para levantar la aplicación con una configuración reutilizable.
- Interpretar logs y solucionar errores frecuentes durante la ejecución.

Desarrollo de contenidos

1. ¿Qué es Docker?

Docker es una plataforma de contenedores. Permite crear imágenes que contienen una aplicación, sus librerías, configuraciones y el comando necesario para iniciar el programa. A partir de una imagen se crean contenedores, que son procesos aislados que ejecutan esa aplicación.

A diferencia de una máquina virtual completa, un contenedor no necesita instalar un sistema operativo completo por cada aplicación. Comparte el kernel del sistema anfitrión y ejecuta solo lo necesario para el proceso de la aplicación. Por eso suele iniciar rápido y usar menos recursos que una máquina virtual tradicional.

Concepto	Explicación sencilla
Imagen	Plantilla de solo lectura con instrucciones y archivos para crear contenedores.
Contenedor	Instancia en ejecución de una imagen. Es donde realmente corre la aplicación.
Dockerfile	Archivo con instrucciones para construir una imagen.
Docker Engine	Motor que crea y administra imágenes, contenedores, redes y volúmenes.
Docker Desktop	Aplicación para Windows/Mac/Linux que integra motor, CLI, interfaz gráfica y Compose.
Docker Compose	Herramienta para definir servicios en un archivo YAML y levantarlos con un solo comando.

2. ¿Para qué se usa Docker?

Docker se usa para desarrollar, probar y desplegar aplicaciones de forma más consistente. En formación, permite que todos los aprendices ejecuten el mismo entorno sin instalar manualmente cada dependencia del proyecto.

- Ejecutar aplicaciones web sin instalar todas las dependencias directamente en Windows.
- Probar una API, una base de datos o un servicio con comandos reproducibles.
- Evitar diferencias entre el equipo del aprendiz, el equipo del instructor y el servidor.
- Preparar proyectos para despliegue en la nube o ambientes DevOps.
- Crear laboratorios de bases de datos, backend, frontend, colas, caché y microservicios.

3. Requisitos previos para instalación en Windows

Para esta guía se trabajará con Docker Desktop en Windows. Antes de instalar, el equipo debe cumplir requisitos mínimos y preferiblemente tener habilitado WSL 2, que permite ejecutar contenedores Linux de forma integrada en Windows.

Requisito	Descripción
Sistema operativo	Windows 10 o Windows 11 compatible con Docker Desktop.
Virtualización	Debe estar habilitada en BIOS/UEFI.
WSL 2	Recomendado para usar contenedores Linux en Windows.
Permisos	Usuario con permisos para instalar aplicaciones y habilitar componentes.
Conexión a internet	Necesaria para descargar Docker Desktop e imágenes base.
Terminal	PowerShell, CMD o Windows Terminal para ejecutar comandos Docker.

Instalación, verificación y uso de Docker Desktop

Instalación en Windows, verificación del motor y ejecución de Dino Run con Docker Compose

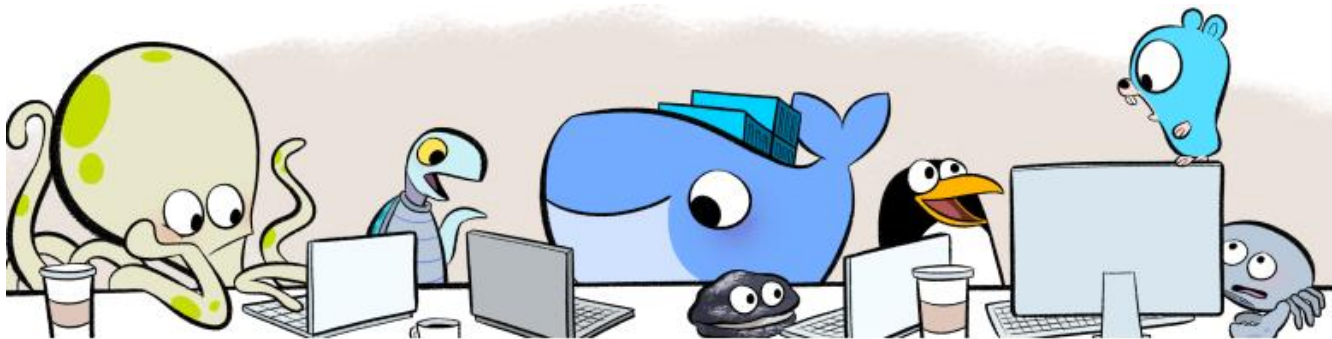
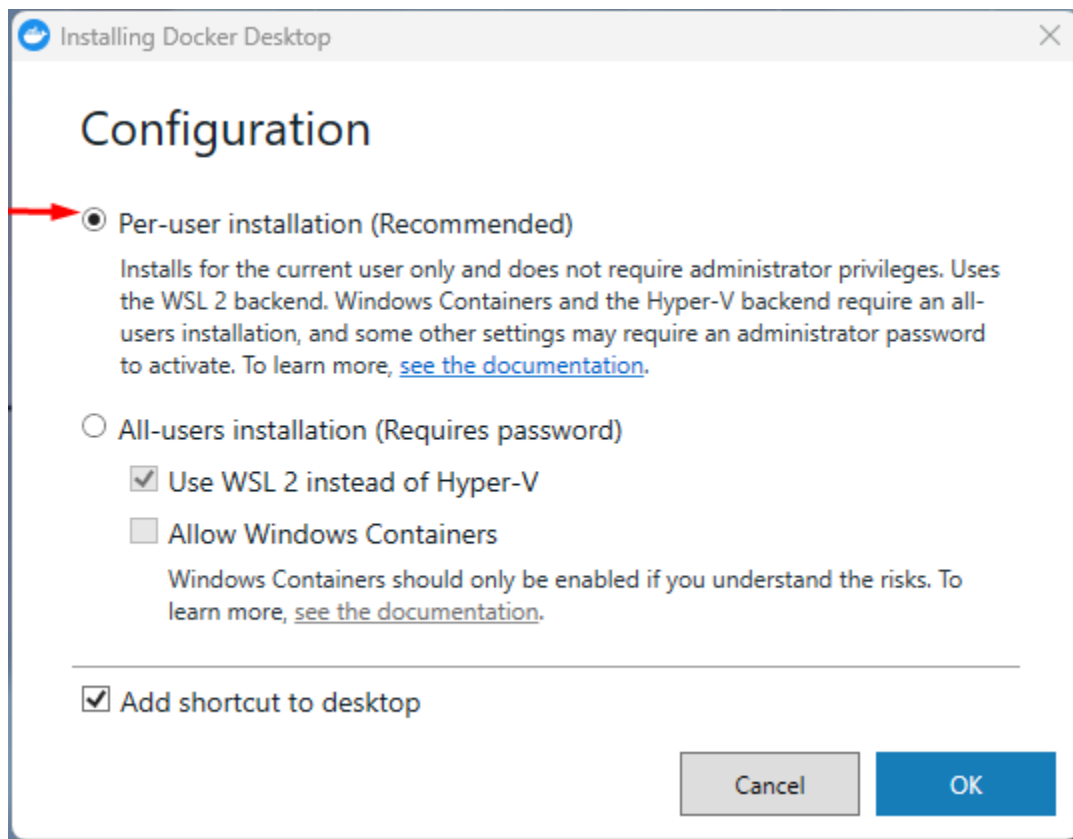
4. Instalar Docker Desktop

1. Ingresar a la página oficial de Docker Desktop.
2. Descargar el instalador para Windows.
3. Ejecutar Docker Desktop Installer.exe.
4. Mantener activa la opción de WSL 2 si el equipo lo permite.
5. Finalizar la instalación y reiniciar el equipo si el instalador lo solicita.
6. Abrir Docker Desktop desde el menú de inicio.
7. Aceptar los términos de uso y esperar a que el motor de Docker quede iniciado.

<https://docs.docker.com/desktop/setup/install/windows-install/>

4.1 Evidencias visuales de la instalación

The screenshot shows the Docker Docs website with the 'Manuales' (Manuals) tab selected. The left sidebar lists various topics under 'IA Y AGENTES' and 'DESARROLLO DE APLICACIONES'. The main content area is titled 'Instalar Docker Desktop en Windows' and includes a 'Términos de Docker Desktop' section, a download link for 'Docker Desktop para Windows - x86_64' (highlighted with a red box), and a 'Modos de instalación' section. The right sidebar contains a 'Tabla de contenido' (Table of contents) with links to various installation steps. A red arrow points to the download link.



Docker Subscription Service Agreement

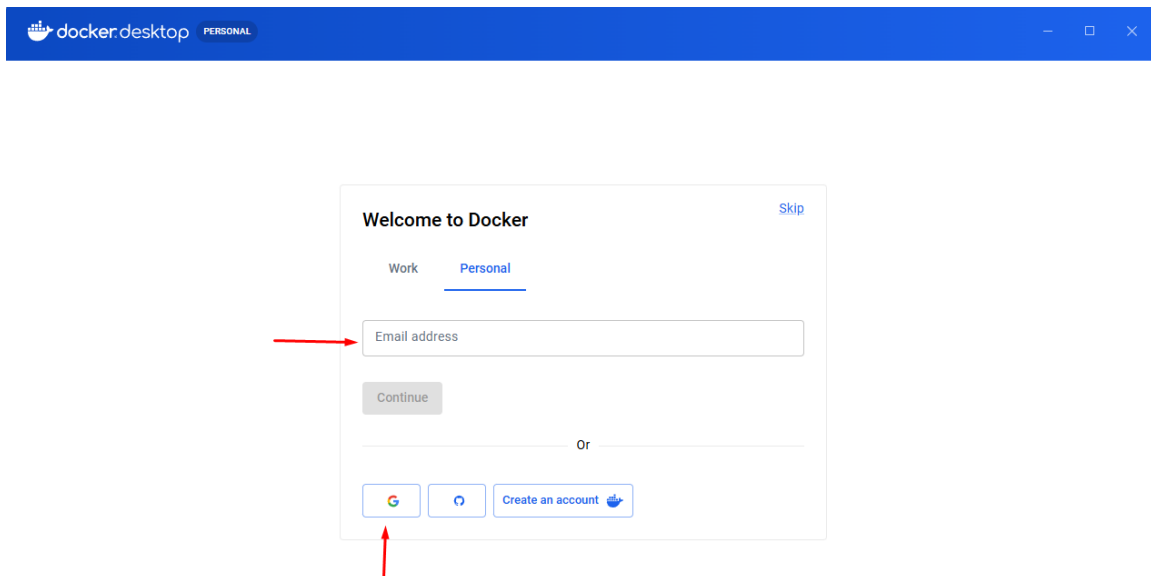
By selecting **accept**, you agree to the [Subscription Service Agreement](#), the [Docker Data Processing Agreement](#), the [Data Privacy Policy](#) and the [Docker AI Supplemental Terms](#).

Commercial use of Docker Desktop at a company of more than 250 employees OR more than \$10 million in annual revenue requires a paid subscription (Pro, Team, or Business). [See subscription details](#)

[View Full Terms](#)

[Accept](#)

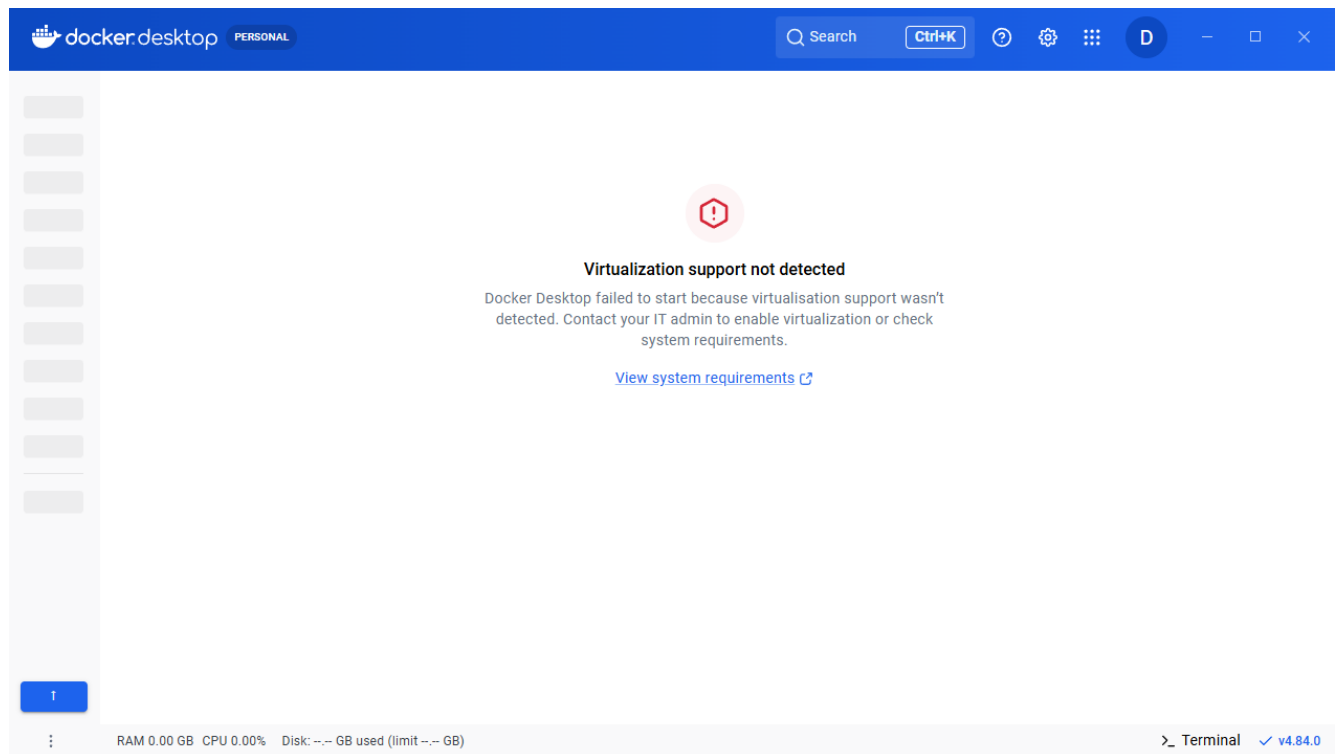
[Close](#)



5. Verificar y completar la instalación

5.1 Instalar WSL 2 cuando no está disponible

Si Docker Desktop informa que WSL no está instalado, abra PowerShell como administrador, ejecute `wsl --install` y reinicie el equipo cuando Windows lo solicite.



```

Administrador: Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\WINDOWS\system32> wsl --update
>>
>> wsl --install
Descargando: Subsistema de Windows para Linux 2.7.11
Instalando: Subsistema de Windows para Linux 2.7.11
Se ha instalado Subsistema de Windows para Linux 2.7.11.
Instalando componente opcional de Windows: VirtualMachinePlatform

Herramienta Administración y mantenimiento de imágenes de implementación
Versión: 10.0.26100.8972

Versión de imagen: 10.0.26200.8973

Habilitando características
[=====100.0%=====]
La operación se completó correctamente.
La operación solicitada se realizó correctamente. Los cambios se aplicarán una vez que se reinicie el sistema.
Comprobando actualizaciones.
La versión más reciente de Subsistema de Windows para Linux ya está instalada.
La operación solicitada se realizó correctamente. Los cambios se aplicarán una vez que se reinicie el sistema.
PS C:\WINDOWS\system32>
  
```

5.2 Habilitar la virtualización en BIOS/UEFI

Si después de instalar WSL 2 Docker Desktop todavía informa que la virtualización está deshabilitada, consulte la documentación del fabricante del equipo y habilite Intel Virtualization Technology, Intel VT-x, AMD-V o SVM Mode en BIOS/UEFI. Las pantallas cambian según el fabricante; las siguientes imágenes muestran solamente un ejemplo de una placa Gigabyte.







5.3 Comprobar Docker Engine y Docker Compose

Una vez abierto Docker Desktop, se debe validar desde terminal que los comandos funcionen. Esta verificación permite confirmar que Docker Engine y la CLI están disponibles.

Comandos de verificación

```
docker --version
docker compose version
docker info
docker run hello-world
```

```

Microsoft Windows [Versión 10.0.26200.8973]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\user>docker --version
Docker version 29.6.2, build dfc4efb

C:\Users\user>docker compose version
Docker Compose version v5.3.1

C:\Users\user>docker info
Client:
Version:      29.6.2
Context:      desktop-linux
Debug Mode:   false
Plugins:
agent: Docker AI Agent Runner (Docker Inc.)
  Version:    v1.111.0
  Path:       C:\Users\user\.docker\cli-plugins\docker-agent.exe
ai: Docker AI Agent - Ask Gordon (Docker Inc.)
  Version:    v1.27.0
  Path:       C:\Users\user\.docker\cli-plugins\docker-ai.exe
buildx: Docker Buildx (Docker Inc.)
  Version:    v0.35.0-desktop.2
  Path:       C:\Users\user\.docker\cli-plugins\docker-buildx.exe
compose: Docker Compose (Docker Inc.)
  Version:    v5.3.1
  Path:       C:\Users\user\.docker\cli-plugins\docker-compose.exe
debug: Get a shell into any image or container (Docker Inc.)
  Version:    0.0.47
  Path:       C:\Users\user\.docker\cli-plugins\docker-debug.exe
desktop: Docker Desktop commands (Docker Inc.)
  Version:    v0.4.3
  Path:       C:\Users\user\.docker\cli-plugins\docker-desktop.exe
dhi: CLI for managing Docker Hardened Images (Docker Inc.)
  Version:    v0.0.7
  Path:       C:\Users\user\.docker\cli-plugins\docker-dhi.exe
extension: Manages Docker extensions (Docker Inc.)
  Version:    v0.2.31
  Path:       C:\Users\user\.docker\cli-plugins\docker-extension.exe
init: Creates Docker-related starter files for your project (Docker Inc.)
  Version:    v1.4.0
  Path:       C:\Users\user\.docker\cli-plugins\docker-init.exe
mcp: Docker MCP Plugin (Docker Inc.)
  Version:    v0.43.3
  Path:       C:\Users\user\.docker\cli-plugins\docker-mcp.exe
model: Docker Model Runner (Docker Inc.)
  Version:    v1.2.6
  Path:       C:\Users\user\.docker\cli-plugins\docker-model.exe
offload: Docker Offload (Docker Inc.)
  Version:    v0.6.9
  Path:       C:\Users\user\.docker\cli-plugins\docker-offload.exe
pass: Docker Pass Secrets Manager Plugin (beta) (Docker Inc.)
  Version:    v0.2.0
  Path:       C:\Users\user\.docker\cli-plugins\docker-pass.exe
sandbox: "docker sandbox" is deprecated, use Docker Sandboxes instead (Docker Inc.)
  
```

Si el comando `docker run hello-world` finaliza correctamente, Docker pudo descargar una imagen de prueba, crear un contenedor y ejecutarlo.

```

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/

C:\Users\user>

```

6. Preparar la aplicación Dino Run

El ejemplo práctico será Dino Run, una aplicación web real creada con Python y Flask. Permite registrar usuarios, iniciar sesión, jugar una carrera de dinosaurio y conservar el récord personal y la clasificación de los diez mejores puntajes. La aplicación usa SQLite para almacenar los datos; por eso se configurará un bind mount entre la carpeta data del equipo y /app/data dentro del contenedor.

Con el fin de analizar el uso de Docker, a continuación se presenta la configuración realizada con Gordon, el asistente de inteligencia artificial de Docker. Gordon ayudó a crear los tres archivos necesarios: Dockerfile, .dockerignore y compose.yaml. La práctica se realizó con el plan base y también incluye enlaces a la documentación oficial para aprender a crear cada archivo manualmente.

6.1 Estructura del proyecto

Estructura recomendada del proyecto

```

sena-docker-flask-app/
├── app.py
├── requirements.txt
├── Dockerfile
├── .dockerignore
├── compose.yaml
├── iniciar_juego.bat
├── templates/
├── static/
├── data/
│   └── dino.db (se genera durante el uso)

```

6.2 Creación de Dockerfile

El Dockerfile es un archivo de texto sin extensión que contiene la receta para construir una imagen de Dino Run. Define la versión de Python, la carpeta de trabajo, las dependencias, los archivos que se copian, el usuario de ejecución, el puerto, la comprobación de salud y el comando de inicio. Compose utiliza este archivo porque build: . le indica que debe construir la imagen desde el código del proyecto.

Dockerfile completo

```
FROM python:3.12-slim

WORKDIR /app

RUN groupadd -r appuser && useradd -r -g appuser appuser

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .
COPY static/ static/
COPY templates/ templates/

RUN chown -R appuser:appuser /app

USER appuser

EXPOSE 5000

HEALTHCHECK --interval=30s --timeout=5s --start-period=10s --retries=3 \
    CMD python -c "import urllib.request;
    urllib.request.urlopen('http://localhost:5000/')" || exit 1

CMD ["python", "app.py"]
```

¿Por qué se crea este Dockerfile y de dónde salen sus instrucciones?

El Dockerfile no es igual para todos los proyectos. Sus instrucciones salen de revisar el lenguaje, las dependencias, la estructura de carpetas, el puerto y el comando de inicio de la aplicación. En Dino Run, app.py demuestra que el lenguaje es Python, requirements.txt declara Flask y Werkzeug, templates y static contienen la interfaz, y app.run indica que el servidor escucha en el puerto 5000.

Método para adaptar un Dockerfile a otro proyecto:

1. Identificar el lenguaje, el framework y una imagen base compatible.
2. Localizar el archivo de dependencias, por ejemplo requirements.txt o package.json.

3. Identificar el comando correcto para iniciar la aplicación.
4. Revisar qué puerto utiliza el servidor en el código.
5. Identificar qué carpetas deben copiarse y cuáles necesitan persistencia.
6. Aplicar seguridad, healthcheck y .dockerignore según las necesidades del proyecto.

Para este proyecto:

FROM python:3.12-slim

Selecciona una imagen oficial ligera que ya contiene Python 3.12 y pip.

Se elige Python porque app.py está escrito en este lenguaje; slim reduce el tamaño base.

WORKDIR /app

Define /app como el directorio de trabajo dentro del contenedor.

Las instrucciones posteriores se ejecutan tomando /app como ubicación actual.

RUN groupadd -r appuser && useradd -r -g appuser appuser

Crea el grupo y el usuario de sistema appuser para no ejecutar la aplicación como root.

Limitar privilegios reduce el impacto de una posible vulnerabilidad.

COPY requirements.txt .

Copia primero requirements.txt para separar la instalación de dependencias del código.

Si solo cambia app.py, Docker puede reutilizar la capa de dependencias desde la caché.

Las dependencias solo se reinstalan cuando cambia requirements.txt.

RUN pip install --no-cache-dir -r requirements.txt

Instala con pip las dependencias declaradas en requirements.txt.

--no-cache-dir evita guardar la caché de pip y ayuda a reducir el tamaño de la imagen.

COPY app.py .

COPY static/ static/

COPY templates/ templates/

Copia el código Python, las plantillas HTML y los archivos estáticos del juego.

Se copian después de las dependencias para aprovechar mejor la caché de construcción.

RUN chown -R appuser:appuser /app

Entrega al usuario appuser la propiedad de los archivos ubicados en /app.

COPY crea los archivos como root; chown permite que appuser trabaje con ellos.

USER appuser

Indica que las instrucciones siguientes y el proceso principal se ejecutan como appuser.

El contenedor ejecuta Dino Run con permisos limitados.

EXPOSE 5000

Documenta que la aplicación escucha internamente en el puerto 5000.

EXPOSE no publica el puerto; ports en compose.yaml realiza la conexión con el computador.

**HEALTHCHECK --interval=30s --timeout=5s --start-period=10s --retries=3 **

CMD python -c "import urllib.request; urllib.request.urlopen('http://localhost:5000/')" || exit 1

Comprueba cada 30 segundos que la página principal responda correctamente.

Docker muestra healthy o unhealthy. Esta prueba no reinicia por sí sola un contenedor unhealthy; sirve para diagnóstico y para dependencias condicionadas por salud.

CMD ["python", "app.py"]

Define el comando predeterminado que inicia app.py cuando se crea el contenedor.

Documentación oficial de Docker:

<https://docs.docker.com/build/concepts/dockerfile/>

<https://docs.docker.com/reference/dockerfile/>

6.3 Creación de .dockerignore

.dockerignore

```
.venv
env/
__pycache__
*.pyc
*.pyo
*.pyd
.Python
.git
.gitignore
.DS_Store
*.log
.env
*.db
data/
.guia_sena_docx/
*.docx
*.bat
~$*
dino_stderr.log
dino_stdout.log
```

¿Por qué se debe crear .dockerignore?

.dockerignore es un archivo de texto que filtra el contexto de construcción. Indica qué archivos y carpetas no deben enviarse al constructor de imágenes. Esto acelera la construcción, reduce el tamaño del contexto y evita incorporar datos locales, secretos o archivos que el contenedor no necesita.

```
.venv/ __pycache__ / *.py[cod] data/ *.db *.log .git/ *.docx
```

En Dino Run se excluye .venv porque pertenece al Python instalado en el computador; data y *.db porque contienen datos persistentes y se conectan después mediante un bind mount; .env porque puede contener secretos; los cachés y logs porque se regeneran; y los documentos y archivos .bat porque no son necesarios dentro de una imagen Linux.

Aunque puede omitirse en un ejercicio muy pequeño, se recomienda usar .dockerignore en proyectos reales. No se deben excluir app.py, requirements.txt, templates ni static, porque el Dockerfile necesita copiarlos. Si COPY informa que un archivo no existe, se debe revisar si algún patrón de .dockerignore lo está excluyendo.

Elemento	Razón
.venv	Entorno virtual del computador; no funciona ni se necesita dentro de la imagen Linux.
__pycache__	Caché compilada de Python; se genera nuevamente cuando es necesaria.
.git/ y .gitignore	Historial y configuración de control de versiones; no son necesarios para ejecutar la aplicación.
*.pyc, *.pyo, *.pyd	Bytecode y extensiones compiladas locales de Python; pueden ser incompatibles con Linux.
.Python	Archivo usado por algunos entornos locales de Python; no es parte de Dino Run.
.env	Puede contener variables sensibles; la configuración se entrega en compose.yaml o mediante secretos.
*.log	Registros locales de desarrollo; el contenedor genera sus propios logs.
*.db	Base de datos local; se conserva en la carpeta data mediante el bind mount y no debe quedar dentro de la imagen.
data/	Directorio persistente; se conecta en tiempo de ejecución mediante el bind mount ./data:/app/data y no debe incorporarse a la imagen.
Archivos .docx, .bat	Documentos de la guía y archivos de inicio de Windows; no se usan dentro del contenedor Linux. Nota: dino_stderr.log y dino_stdout.log ya coinciden con el patrón *.log. Repetirlos no produce un error, pero se mantienen en esta práctica para mostrar explícitamente que los registros locales no se copian.

Documentación oficial de Docker:

<https://docs.docker.com/build/concepts/context/>

7. Creación y comprensión de compose.yaml

compose.yaml

```
services:
  app:
    build: .
    container_name: dino-game
    ports:
      - "5000:5000"
    volumes:
      - ./data:/app/data
    environment:
      - DATABASE_PATH=/app/data/dino.db
      - SECRET_KEY=tu-clave-secreta-mejor-cambiar-en-produccion
      - FLASK_ENV=production
    restart: unless-stopped
    healthcheck:
```

```
test: ["CMD", "python", "-c", "import urllib.request;
urllib.request.urlopen('http://localhost:5000/')"]
interval: 30s
timeout: 5s
start_period: 10s
retries: 3
```

7.1 Qué es Docker Compose

Docker Compose es una herramienta que permite describir cómo se debe construir y ejecutar una aplicación formada por uno o varios contenedores.

Este archivo utiliza el formato YAML y debe guardarse en la carpeta principal del proyecto, junto al Dockerfile.

Para Dino Run, Compose guarda en un solo lugar:

- Cómo construir la imagen.
- El nombre del contenedor.
- Los puertos.
- La carpeta donde se almacena SQLite.
- Las variables de entorno.
- La política de reinicio.
- La comprobación del estado de la aplicación.

Documentación oficial de Docker:

<https://docs.docker.com/compose/>

7.2 Por qué utilizar compose.yaml

No es obligatorio. La aplicación podría ejecutarse configurando manualmente el contenedor desde Docker Desktop o escribiendo un comando docker run.

Sin embargo, compose.yaml es recomendable porque evita repetir la configuración cada vez. También permite que otra persona ejecute el proyecto con los mismos puertos, rutas, variables y condiciones.

Configuración manual sin Compose

configurar manualmente imagen + puerto + volumen + variables + reinicio

Configuración declarada con Compose

la configuración queda documentada en compose.yaml

7.3 Explicación de las instrucciones

Instrucción	Explicación
services:	Inicia la lista de programas o servicios que administrará Compose.
app:	Nombre interno del servicio. En este proyecto representa la aplicación Flask.
build: .	Indica que la imagen debe construirse usando el Dockerfile ubicado en la carpeta actual. El punto significa “esta carpeta”.
container_name: dino-game	Asigna al contenedor el nombre visible dino-game.
ports:	Inicia la configuración de puertos.
"5000:5000"	Conecta el puerto 5000 del computador con el puerto 5000 del contenedor.
volumes:	Declara montajes de almacenamiento. En este proyecto contiene un bind mount de una carpeta local.
./data:/app/data	Conecta directamente la carpeta local data con /app/data dentro del contenedor para conservar usuarios y puntuaciones.
environment:	Define variables de entorno disponibles para la aplicación.
DATABASE_PATH=/app/data/dino.db	Le indica a Dino Run dónde debe guardar la base de datos SQLite.
SECRET_KEY=...	Clave utilizada por Flask para proteger las sesiones. El valor mostrado es únicamente de práctica; en producción debe suministrarse como secreto externo.
FLASK_ENV=production	Variable opcional que app.py no consulta; Flask moderno no la necesita para ejecutar Dino Run.
restart: unless-stopped	Reinicia el contenedor si termina, excepto cuando el usuario lo detiene de manera explícita.
healthcheck:	Comprueba periódicamente que la página principal responda y permite mostrar el estado healthy o unhealthy.

Documentación oficial de Docker:

<https://docs.docker.com/compose/intro/compose-application-model/>

Cuando Compose procesa este archivo:

1. Lee el Dockerfile.
2. Construye la imagen.
3. Crea el contenedor dino-game.
4. Publica el puerto 5000.
5. Conecta la carpeta data.

6. Entrega las variables de entorno a Flask.
7. Inicia python app.py.
8. Comprueba periódicamente la salud de la aplicación.
9. Muestra el proyecto en Docker Desktop como un grupo de Compose.

El archivo compose.yaml no contiene el programa ni reemplaza el Dockerfile: organiza cómo se ejecutará la imagen creada por el Dockerfile.

8. Construir y revisar el proyecto con Docker Compose

Con Dockerfile, .dockerignore y compose.yaml listos, el procedimiento recomendado es usar Docker Compose como único flujo. Desde Gordon se selecciona la carpeta del proyecto y se solicita construir e iniciar la aplicación. Compose lee el servicio app, utiliza el Dockerfile, crea la imagen sena-docker-flask-app-app:latest y ejecuta el contenedor dino-game con el puerto, el montaje de datos, las variables y la comprobación de salud definidos en compose.yaml.

8.1 Construir con Gordon y Docker Compose

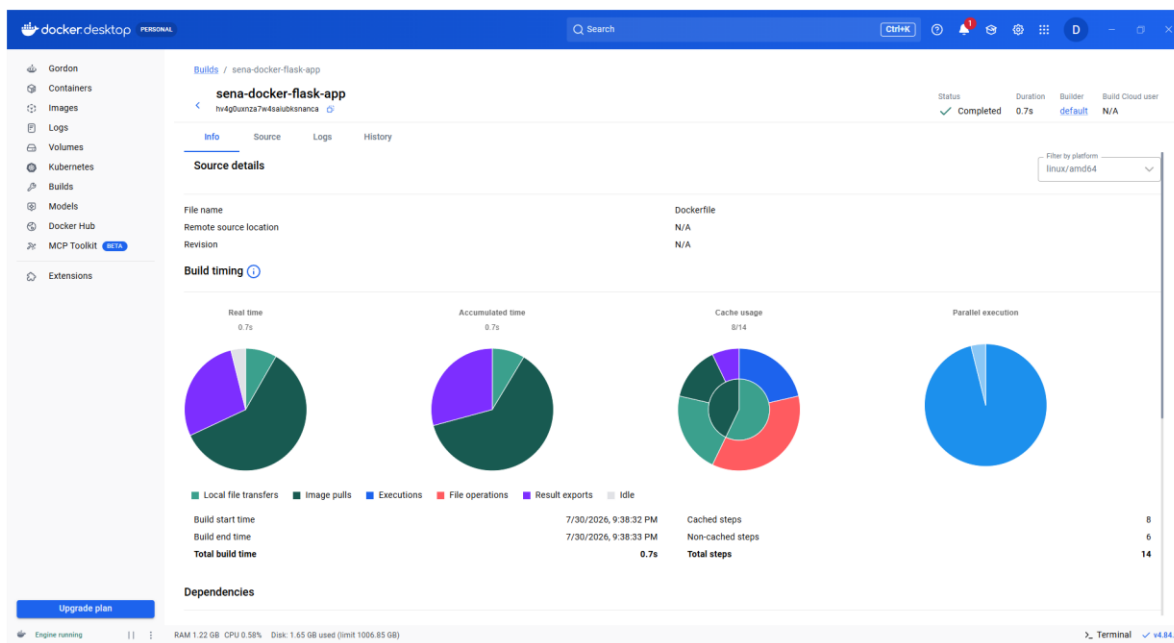
En Gordon se puede escribir: «Revisa Dockerfile, .dockerignore y compose.yaml. Si son coherentes, construye e inicia el proyecto con Docker Compose. Explícame cada acción antes de ejecutarla». Gordon utiliza las herramientas de Docker Desktop y muestra el resultado en la sesión.

```
docker compose up -d --build
```

Esta es la acción equivalente que realiza Docker: --build construye la imagen antes de iniciar; -d deja el servicio funcionando en segundo plano. El aprendiz no necesita memorizarla para usar Gordon, pero conocerla ayuda a comprender qué sucede detrás de la interfaz.

8.2 Verificar la construcción en Builds

Abra Builds en el menú izquierdo, seleccione la construcción más reciente de sena-docker-flask-app y confirme Status: Completed. En Info se revisan duración y uso de caché; Source muestra el Dockerfile; Logs enseña cada instrucción ejecutada.



8.3 Verificar la imagen en Images

El nombre sena-docker-flask-app-app se forma con el nombre del proyecto sena-docker-flask-app y el servicio app declarado en compose.yaml. Como el servicio define build: . y no define image:, Compose asigna automáticamente el nombre project-service.

El punto verde indica que la imagen está en uso. Name identifica el repositorio local; Tag muestra la etiqueta latest; Image ID identifica la imagen; Created indica cuándo se construyó; Size muestra su tamaño aproximado; y Actions ofrece operaciones adicionales.

The screenshot shows the Docker Desktop interface with the 'Images' tab selected. The 'Local' tab is active, showing a table of local images. The table has columns for Name, Tag, Image ID, Created, Size, and Actions. One image is listed: 'sena-docker-flask-app-app' with tag 'latest' and image ID '38f391501f80'. The 'Created' column shows '12 hours ago' and the 'Size' column shows '197.35 MB'. The 'Actions' column contains icons for refresh, play, and delete.

Name	Tag	Image ID	Created	Size	Actions
sena-docker-flask-app-app	latest	38f391501f80	12 hours ago	197.35 MB	Refresh, Play, Delete

Relación entre los elementos

1. Gordon y compose.yaml
2. Builds: registra la construcción
3. Images: guarda sena-docker-flask-app-app
4. Containers: ejecuta dino-game
5. Puerto 5000: abre Dino Run en el navegador

[Documentación oficial de Docker: vista Builds de Docker Desktop](#)

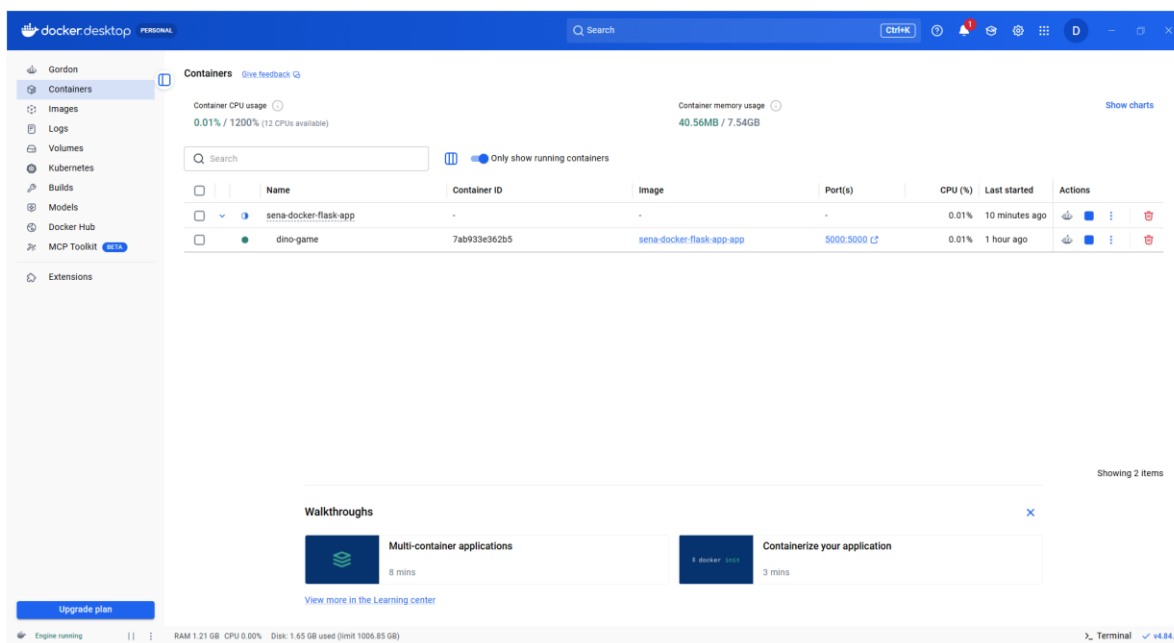
[Documentación oficial de Docker: vista Images de Docker Desktop](#)

[Documentación oficial de Docker: docker compose up](#)

9. Ejecutar y comprobar Dino Run con Docker Compose

Al finalizar la acción de Gordon, Docker Compose ya creó e inició el contenedor con toda la configuración de compose.yaml. No se debe pulsar Run desde Images, porque esa opción crearía un contenedor independiente y obligaría a repetir manualmente el puerto, el montaje y las variables. Para comprobar el resultado, abra Containers y despliegue el grupo sena-docker-flask-app.

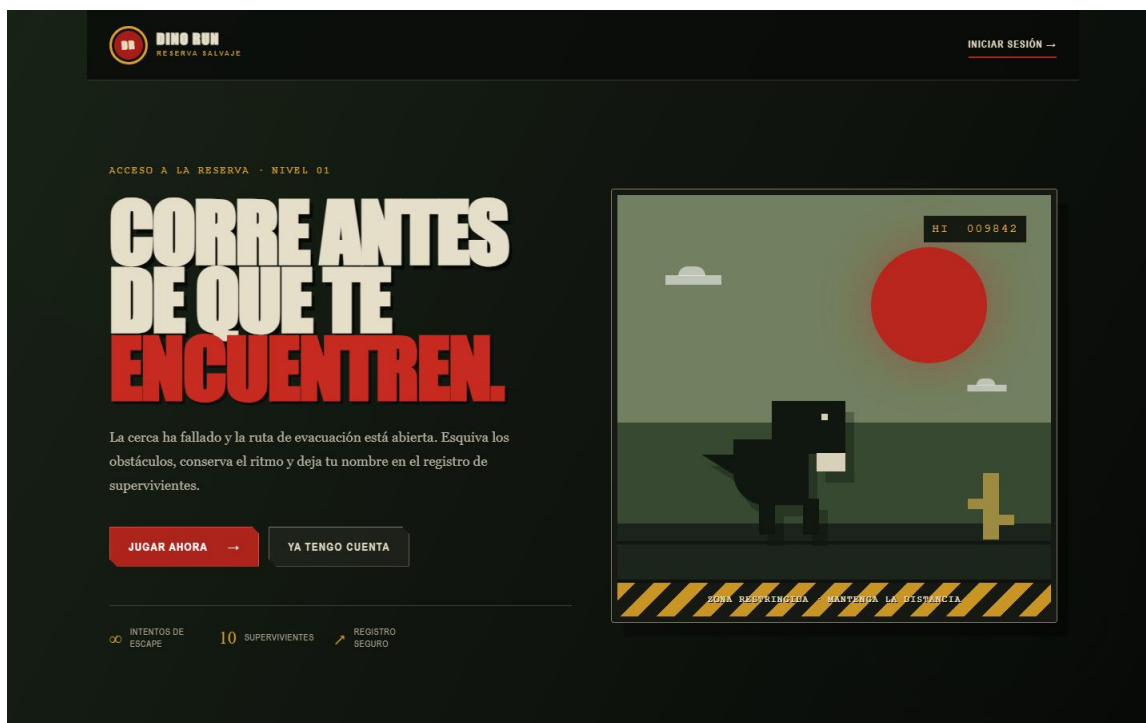
9.1 Verificar el contenedor en Containers



Comprobar el contenedor desde Docker Desktop

1. Abra Containers. 2. Despliegue sena-docker-flask-app. 3. Confirme que dino-game tenga punto verde y estado Running o Healthy. 4. Verifique que la imagen sea sena-docker-flask-app-app. 5. Pulse el enlace 5000:5000 para abrir <http://localhost:5000>.

9.2 Abrir la aplicación desde el puerto publicado



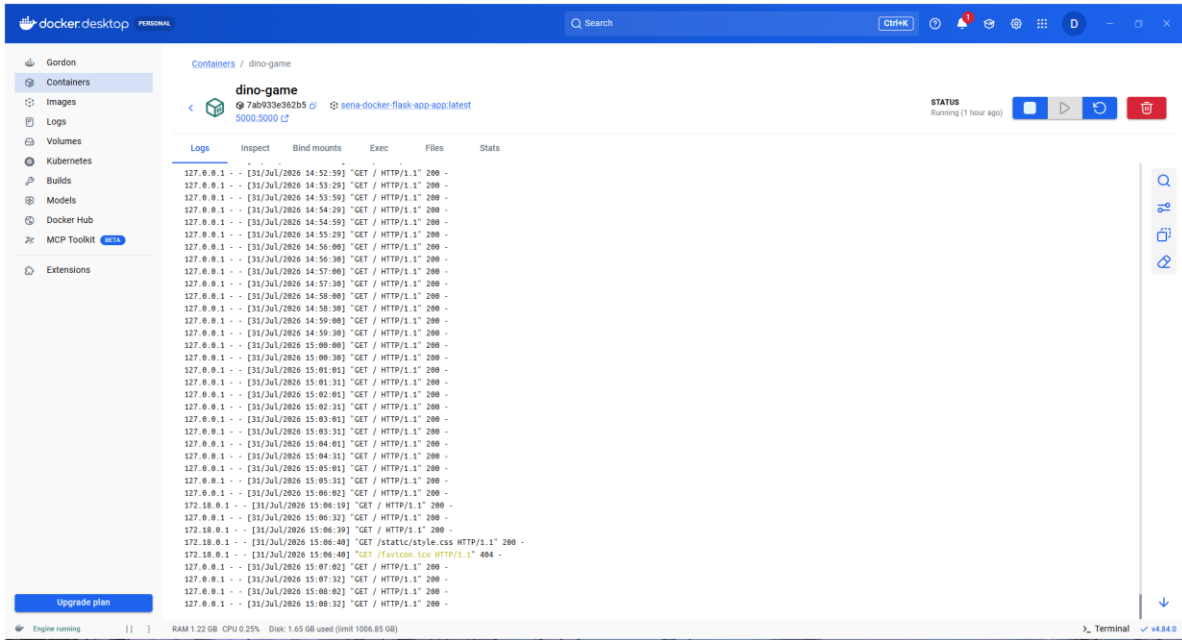
[Documentación oficial de Docker: vista Containers de Docker Desktop](#)

10. Administrar y diagnosticar el contenedor

La vista Containers concentra las operaciones diarias. La fila sena-docker-flask-app representa la aplicación de Compose y la fila dino-game representa el contenedor del servicio app. Seleccione dino-game para abrir Logs, Inspect, Bind mounts, Exec, Files y Stats.

Acción	Ruta dentro de Docker Desktop
Iniciar, detener o reiniciar	Containers → sena-docker-flask-app o dino-game → botones Start, Stop o Restart.
Consultar registros	Containers → dino-game → pestaña Logs.
Revisar puertos y configuración	Containers → dino-game → pestaña Inspect.
Explorar archivos del contenedor	Containers → dino-game → pestaña Files.
Abrir la aplicación	Containers → dino-game → enlace 5000:5000.
Revisar persistencia	Containers → dino-game → pestaña Bind mounts; confirme data → /app/data.
Reconstruir después de cambios	Vuelva a Gordon, solicite construir e iniciar con Compose y después revise Builds e Images.

10.1 Consultar los registros de ejecución

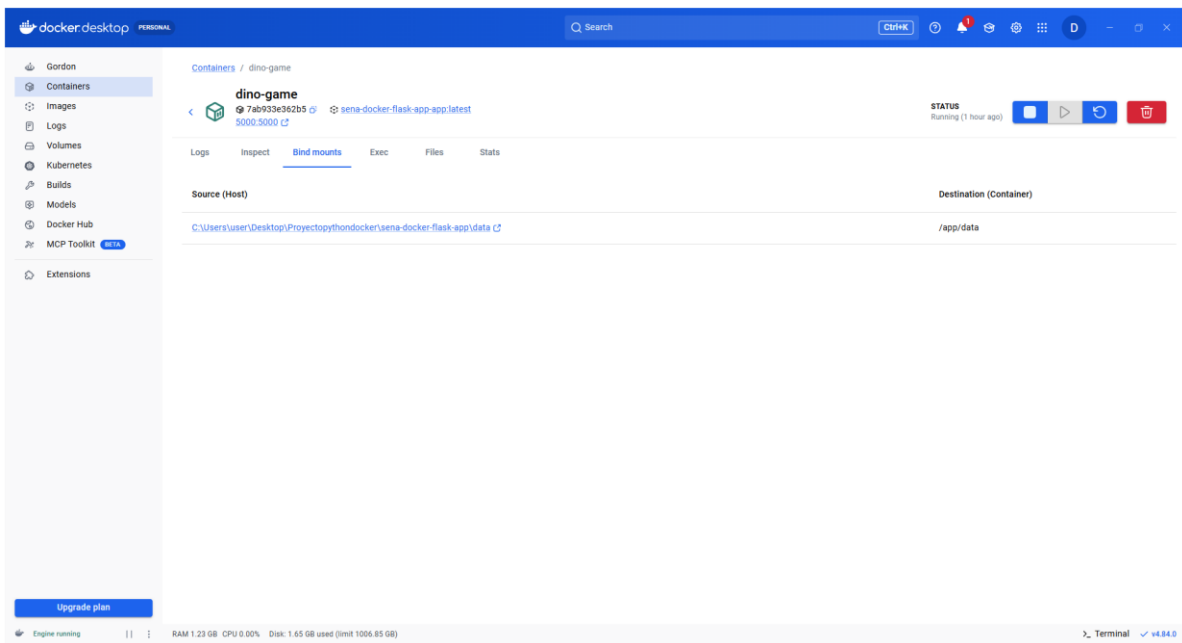


[Documentación oficial de Docker: inspeccionar contenedores y logs](#)

11. Verificar la persistencia mediante un bind mount

Dino Run guarda usuarios y puntuaciones en SQLite, dentro de `/app/data/dino.db`. Para que esos datos permanezcan fuera del ciclo de vida del contenedor, `compose.yaml` define el bind mount `./data:/app/data`. La carpeta `data` del proyecto es la fuente del computador y `/app/data` es el destino dentro del contenedor.

11.1 Comprobar el bind mount en Docker Desktop



Procedimiento definido por compose.yaml

1. Confirme en compose.yaml la sección volumes con `./data:/app/data`.
2. Solicite a Gordon construir e iniciar el proyecto con Docker Compose.
3. Abra Containers `dino-game` Bind mounts.
4. Confirme la ruta local data como Source y `/app/data` como Destination.
5. Registre un usuario, obtenga una puntuación y reinicie el contenedor para comprobar la persistencia.

Comprobación desde Docker Desktop

En Containers `dino-game` Bind mounts debe aparecer la carpeta local data conectada con `/app/data`. Este proyecto utiliza un bind mount, no un volumen administrado por Docker; por eso la evidencia principal se consulta en Bind mounts y no necesita aparecer en la vista Volumes.

Resultado esperado: después de reiniciar dino-game, el archivo `data/dino.db` permanece en la carpeta del proyecto y los usuarios y puntuaciones continúan disponibles.

[Documentación oficial de Docker: bind mounts](#)

12. Comprender y adaptar las variables de entorno

Las variables de entorno de este proyecto se entregan desde `compose.yaml` al contenedor; no es necesario escribir instrucciones `ENV` en el `Dockerfile`. `app.py` lee `DATABASE_PATH` y `SECRET_KEY` con `os.getenv`. Para la práctica local existen valores predeterminados, pero una publicación real debe proporcionar una `SECRET_KEY` privada fuera del código y del documento.

Variables utilizadas por Dino Run

`DATABASE_PATH=/app/data/dino.db`: coincide con la ruta usada por SQLite y con el bind mount; puede omitirse porque `app.py` tiene el mismo valor predeterminado, pero dejarla explícita documenta la configuración. `SECRET_KEY`: Flask la utiliza para proteger las sesiones; el valor de práctica permite ejecutar localmente, pero en producción debe sustituirse por un secreto largo y privado. `FLASK_ENV=production`: `app.py` no consulta esta variable y Flask moderno ya no la necesita; puede omitirse en este proyecto. `PYTHONDONTWRITEBYTECODE` y `PYTHONUNBUFFERED`: son ajustes opcionales de Python y no son necesarios para que Dino Run funcione.

Para adaptar variables en otro proyecto:

1. busque `os.getenv`, `process.env`, `System.getenv` u otro mecanismo equivalente en el código;
2. consulte la documentación del framework;
3. identifique valores de configuración y secretos;
4. declare en `compose.yaml` solamente los valores que la aplicación realmente lee;
5. para secretos use un archivo `.env` local excluido por `.dockerignore` y por Git, o un gestor de secretos;
6. solicite a Gordon recrear el proyecto con Compose. Nunca publique claves reales en el `Dockerfile`, `compose.yaml`, capturas o repositorios.

[Documentación oficial de Docker: variables de entorno en Compose](#)

13. Resolver errores frecuentes desde Docker Desktop

Antes de cambiar archivos o volver a construir, siga este orden: Builds confirma la construcción; Images confirma la imagen; Containers muestra el grupo y el contenedor; Logs explica la ejecución; Bind mounts verifica la persistencia; y el enlace 5000:5000 comprueba el acceso desde el navegador.

[Documentación oficial de Docker: vista Builds de Docker Desktop](#)

Secuencia recomendada de diagnóstico: Builds → Images → Containers → Logs → Bind mounts → navegador.

Problema	Causa probable	Solución
Docker Desktop no inicia	El motor o WSL 2 todavía no están disponibles.	Abra Docker Desktop y espere el estado Running; use Troubleshoot si no inicia.
Port is already allocated	Otro proceso o contenedor está usando el puerto 5000 del computador.	Revise Containers y detenga el servicio que ocupa el puerto, o cambie en compose.yaml el puerto del host a "5001:5000" y solicite a Gordon recrear el proyecto.
La imagen sena-docker-flask-app no aparece	La construcción de Compose no terminó o Gordon trabajó sobre otra carpeta.	Abra Builds, seleccione el intento más reciente, revise Error o Logs y vuelva a construir desde la carpeta que contiene compose.yaml.
ModuleNotFoundError: flask	Flask falta en requirements.txt o la imagen quedó desactualizada.	Confirme Flask==3.1.2 y Werkzeug==3.1.8 en requirements.txt; después solicite reconstruir con Compose y revise Builds.
La página no abre	El contenedor está detenido, el puerto no fue publicado o Flask produjo un error.	Revise Containers → dino-game: estado, enlace 5000:5000 y Logs. Confirme también que app.py use host="0.0.0.0".

14. Consideraciones finales

- Docker no reemplaza las buenas prácticas de programación. Si la aplicación falla localmente, probablemente también fallará dentro del contenedor.
- Use imágenes oficiales y versiones específicas cuando sea posible.
- No suba archivos .env, contraseñas ni tokens al repositorio.
- Mantenga el Dockerfile simple y ordenado.
- Mantenga un archivo .dockerignore para evitar enviar entornos virtuales, cachés, bases de datos, documentos y secretos durante la construcción.
- Documente comandos, puertos, variables y resultados de prueba.
- Use Docker Compose cuando necesite repetir de forma consistente la construcción y la ejecución, incluso si el proyecto comienza con un solo servicio.
- En producción, valide seguridad, actualizaciones, vulnerabilidades de imágenes, usuarios no root, logs y monitoreo.

15. Procedimiento general

Fase	¿Qué realiza el aprendiz?
1. Contextualización	Reconoce qué es Docker, para qué se usa y cuáles son sus componentes principales.
2. Instalación	Descarga Docker Desktop, instala con WSL 2 y abre el Dashboard.
3. Verificación inicial	Comprueba en el Dashboard que Docker Desktop indique Engine running y ejecuta una imagen de prueba.
4. Preparación del proyecto	Usa el proyecto Dino Run existente y agrega Dockerfile, compose.yaml y .dockerignore.
5. Construcción guiada	Solicita a Gordon construir e iniciar el proyecto con Compose y revisa el Build completado.
6. Ejecución gráfica	Verifica en Images la imagen sena-docker-flask-app-app y en Containers el grupo y el contenedor dino-game.
7. Validación	Abre http://localhost:5000 , registra un usuario, juega una partida y revisa los logs.
8. Administración	Usa Containers, Images, Builds, Logs y Bind mounts para comprobar y administrar la aplicación.

15.1 Secuencia paso a paso

1. Verificar que Windows tenga virtualización habilitada y conexión a internet.
2. Descargar Docker Desktop desde la página oficial.
3. Ejecutar el instalador y conservar la opción WSL 2 si está disponible.
4. Reiniciar el equipo si el instalador lo solicita.
5. Abrir Docker Desktop y esperar a que el motor esté activo.
6. Confirmar en el Dashboard que Docker Desktop muestre Running.
7. Desde Images, buscar y ejecutar una imagen de prueba para validar el motor.
8. Crear la carpeta sena-docker-flask-app.
9. Verificar que estén app.py, templates, static, requirements.txt, Dockerfile, compose.yaml y .dockerignore.
10. En Gordon, seleccionar la carpeta del proyecto y solicitar construir e iniciar la aplicación con Docker Compose.
11. Abrir Builds y comprobar que la construcción más reciente de sena-docker-flask-app tenga estado Completed.
12. Abrir Images y comprobar sena-docker-flask-app-app:latest marcada en uso.
13. Abrir Containers, desplegar sena-docker-flask-app y comprobar dino-game en estado Running o Healthy.
14. Pulsar 5000:5000, abrir Dino Run y comprobar que responde en el navegador.
15. Abrir dino-game - Logs y confirmar respuestas HTTP 200 sin errores de Python.
16. Abrir dino-game - Bind mounts y confirmar la conexión data - /app/data.
17. Practicar Stop, Start y Restart desde Containers, validar la persistencia y documentar Builds, Images, Containers, aplicación, Logs y Bind mounts.

16. Ejercicios prácticos

Ejercicio 1. Verificación inicial desde Docker Desktop

Problema: comprobar mediante la GUI que el motor funciona y puede ejecutar imágenes.

Procedimiento

Abrir Docker Desktop - comprobar Running - entrar a Images - Search images to run -descargar una imagen de prueba -Run - comprobarla en Containers.

Ejercicio 2. Construcción y revisión visual de la imagen

Problema: construir Dino Run y comprender el resultado desde Images y Builds.

Procedimiento

Gordon - seleccionar la carpeta del proyecto - solicitar construir e iniciar con Docker Compose - Builds - abrir la construcción más reciente - confirmar Completed - revisar Source y Logs - Images - comprobar sena-docker-flask-app-app:latest.

Ejercicio 3. Ejecución y comprobación con Docker Compose

Problema: ejecutar Dino Run mediante Docker Compose y acceder desde el navegador sin configurar manualmente cada opción de docker run.

Procedimiento

Gordon - iniciar con Docker Compose - Containers - desplegar sena-docker-flask-app - confirmar dino-game Running o Healthy - pulsar 5000:5000 - validar Dino Run en el navegador.

Ejercicio 4. Configuración opcional de SECRET_KEY

Problema: distinguir las variables predeterminadas, opcionales y sensibles, y definir las correctamente para Docker Compose.

Procedimiento

Revisar app.py para identificar SECRET_KEY - definirla mediante interpolación desde un archivo .env local no publicado - solicitar a Gordon recrear el proyecto con Compose - comprobar en Inspect solamente que la variable existe, sin mostrar ni capturar su valor real.

Ejercicio 5. Persistencia con bind mount

Problema: conservar usuarios y puntuaciones mediante el bind mount declarado en compose.yaml.

Procedimiento

Confirmar en compose.yaml ./data:/app/data - iniciar con Compose - Containers - dino-game - Bind mounts - registrar un usuario y jugar - Restart - comprobar que el récord permanece.

Ejercicio 6. Diagnóstico visual

Problema: diagnosticar un puerto ocupado y adaptar el puerto del host en compose.yaml.

- Revise el mensaje Port is already allocated en Builds, Gordon o Containers.
- Abra Containers e identifique qué servicio utiliza el puerto 5000.
- Cambie temporalmente en compose.yaml "5000:5000" por "5001:5000" y solicite a Gordon recrear el proyecto.
- Abra <http://localhost:5001>, documente el resultado y restaure después el puerto original.

17. Actividades de aprendizaje

17.1 Reflexión inicial

- Explique con sus palabras qué significa empaquetar una aplicación.
- Mencione un caso donde una aplicación funcione en un equipo pero falle en otro.

17.2 Contextualización y conocimientos previos

- Diferencie imagen, contenedor y Dockerfile.
- Diferencie las vistas Images, Containers y Builds, y las pestañas Logs y Bind mounts.

17.3 Apropiación del conocimiento

- Construya la imagen y estudie sus capas y registros desde Docker Desktop.
- Ejecute Dino Run con Docker Compose y verifique el bind mount data → /app/data.
- Registre un usuario, juegue una partida, reinicie el contenedor y confirme la persistencia del récord.
- Cambie un elemento visual del juego y vuelva a construir la imagen.

18. Evidencias de aprendizaje

18.1 Evidencias de conocimiento

- Definición de Docker, imagen, contenedor, Dockerfile, puerto, volumen, bind mount y Docker Compose.
- Explicación del flujo: construir imagen, ejecutar contenedor y validar aplicación.

18.2 Evidencias de desempeño

- Captura de Docker Desktop instalado y activo.
- Capturas de Gordon, Builds, Images, Containers, aplicación, Logs y Bind mounts.

18.3 Evidencias de producto

- Carpeta del proyecto con app.py, requirements.txt, Dockerfile y compose.yaml.

Carpeta del proyecto Dino Run con app.py, templates, static, requirements.txt, Dockerfile, compose.yaml y .dockerignore.

- Documento de entrega con capturas y explicación de errores corregidos.

19. Criterios de evaluación

- Instala Docker Desktop y se orienta correctamente en su interfaz gráfica.
- Construye la imagen mediante Docker Compose y analiza el proceso desde Builds.
- Ejecuta y comprueba el proyecto mediante Docker Compose, Containers y el puerto publicado.
- Distingue variables opcionales, predeterminadas y sensibles.
- Configura y verifica la persistencia mediante el bind mount declarado en compose.yaml.
- Interpreta logs y corrige errores comunes.
- Entrega evidencias completas sin exponer información sensible.

20. Glosario

Término	Definición
Docker	Plataforma para crear, distribuir y ejecutar aplicaciones en contenedores.
Contenedor	Proceso aislado que ejecuta una aplicación a partir de una imagen.
Imagen	Plantilla con archivos, dependencias y configuración para crear contenedores.
Dockerfile	Archivo de instrucciones para construir una imagen Docker.
Docker Desktop	Aplicación gráfica y herramienta de instalación para usar Docker en Windows, macOS o Linux.
Docker Engine	Motor que administra imágenes, contenedores, redes y volúmenes.
Docker Compose	Herramienta que permite definir y ejecutar servicios desde un archivo YAML.
Puerto	Número lógico usado para acceder a una aplicación o servicio de red.
Variable de entorno	Valor de configuración leído por la aplicación sin estar escrito directamente en el código.
Volumen	Mecanismo para persistir datos fuera del ciclo de vida del contenedor.
Bind mount	Conexión directa entre una carpeta o archivo del equipo anfitrión y una ruta dentro del contenedor.
Red Docker	Canal de comunicación entre contenedores o entre contenedores y el equipo anfitrión.
Build	Proceso de construcción de una imagen.
Run	Proceso de creación y ejecución de un contenedor.
Logs	Registros generados por la aplicación o el contenedor para diagnosticar su estado.
WSL 2	Subsistema de Windows para Linux usado por Docker Desktop para ejecutar contenedores Linux en Windows.

21. Bibliografía

Docker Docs. (2026). What is Docker? / Docker overview. <https://docs.docker.com/get-started/docker-overview/>

Docker Docs. (2026). Install Docker Desktop on Windows. <https://docs.docker.com/desktop/setup/install/windows-install/>

Docker Docs. (2026). Docker Desktop WSL 2 backend on Windows. <https://docs.docker.com/desktop/features/wsl/>

Docker Docs. (2026). Dockerfile reference. <https://docs.docker.com/reference/dockerfile/>

Docker Docs. (2026). Docker Compose Quickstart. <https://docs.docker.com/compose/gettingstarted/>

Docker Docs. (2026). Publishing and exposing ports. <https://docs.docker.com/get-started/docker-concepts/running-containers/publishing-ports/>

Docker Docs. (2026). What is a container? <https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-container/>

Docker Docs. (2026). docker compose up. <https://docs.docker.com/reference/cli/docker/compose/up/>

Docker Docs. (2026). Explore the Builds view in Docker Desktop. <https://docs.docker.com/desktop/use-desktop/builds/>

Docker Docs. (2026). Explore the Images view in Docker Desktop. <https://docs.docker.com/desktop/use-desktop/images/>

Docker Docs. (2026). Explore the Containers view in Docker Desktop. <https://docs.docker.com/desktop/use-desktop/container/>

Docker Docs. (2026). Bind mounts. <https://docs.docker.com/engine/storage/bind-mounts/>

Docker Docs. (2026). Environment variables in Compose. <https://docs.docker.com/compose/how-tos/environment-variables/>

Wired. (2013). The Man Who Would Build a Computer the Size of the Entire Internet. <https://www.wired.com/2013/09/docker>

Servicio Nacional de Aprendizaje (SENA). (s. f.). Material de formación: desarrollo de software, programación web y evidencias de aprendizaje.

FAVA - Formación en Ambientes Virtuales de Aprendizaje

SENA - Servicio Nacional de Aprendizaje.